

Unit Testing Report 2

Button Class

Project: Pygame Tetris Button

Date: 20-Nov-2025

Test Type: White-box (Destructive testing / input validation)

Total Tests: 10

Passed: 7

Failed: 3

1. Summary of Test Failures

test_negative_width_breaks

Expected: ValueError when width < 0

Actual: No error; pygame.Rect allows negative dimensions

Explanation: Pygame does not restrict negative dimensions in Rect creation.

Recommendation: Add a validation check in Button.__init__ to enforce width > 0.

test_insane_size_breaks

Expected: pygame.error when width is extremely large

Actual: No error; pygame can handle huge rects (draw may go off-screen)

Explanation: Pygame does not throw an error for large Rects; they are just drawn off-screen.

Recommendation: Implement a maximum size limit in Button.__init__.

test_invalid_hover_color_type

Expected: TypeError when hover_color is a string

Actual: No error; pygame accepts string colors (e.g., "red")

Explanation: Pygame can interpret string color names, so TypeError is not raised.

Recommendation: Add type checking for hover_color if strict tuple format is required.

2. Detailed Test Table

Test Method	Status	Notes
test_button_none_color_breaks	Pass	pygame.draw.rect fails when color=None
test_invalid_font_breaks	Pass	font.render fails when font=None
test_hover_breaks_with_invalid_mouse	Pass	collidepoint(None) triggers TypeError
test_missing_screen_width_breaks	Pass	SCREEN_WIDTH=None triggers TypeError
test_is_clicked_breaks_without_event_pos	Pass	event.pos missing → AttributeError
test_non_string_text_breaks	Pass	font.render fails with non-string text
test_negative_width_breaks	Fail	pygame.Rect allows negative width
test_insane_size_breaks	Fail	Pygame accepts huge rects, no error raised
test_invalid_hover_color_type	Fail	Pygame accepts string color, no error raised
test_draw_breaks_with_none_surface	Pass	pygame.draw.rect(None, ...) → TypeError

3. Observations

7/10 tests passed (70%), 3/10 tests failed (30%).

Failures occurred due to assumptions about Pygame behavior, not actual code crashes.

Testing type: White-box testing (focused on internal logic and input validation).

Shape Class

All Shape tests passed (100% success).

The tests confirmed correct initialization, rotation handling, movement (left/right/down), and boundary collision behavior. The Shape class correctly cycles through rotation states, prevents invalid moves, and interacts safely with grid limits. No errors, exceptions, or unexpected behaviors were detected. The Shape class is stable and requires no modifications.

Board Class

All Board tests passed (100% success).

Testing verified proper board initialization, grid structure, shape placement, movement validation through `can_move()`, and full row clearing with correct downward shifting. Boundary and occupancy checks behaved consistently, and no out-of-range errors occurred. The Board class logic is reliable, robust, and fully correct under all tested scenarios.

